

Recent advancements have focused on improving SAE performance and addressing limitations. [Rajamanohan et al. \(2024\)](#) introduced a gating mechanism to separate the functionalities of determining which directions to use and estimating their magnitudes, mitigating shrinkage – the systematic underestimation of feature activations. An alternative approach by [Dunefsky et al. \(2024\)](#) uses transcoders to faithfully approximate a densely activating MLP layer with a wider, sparsely-activating MLP layer, offering another path to interpretable feature discovery, a type of sparse distillation ([slavachalnev, 2024](#)).

Sparse Dictionary Learning

Sparse autoencoders ([Cunningham et al., 2024](#)) are proposed as a solution to *polysemantic* neurons. The problem of *superposition* is mathematically formalized as *sparse dictionary learning* ([Olshausen & Field, 1997](#)) problem to decompose neural network activations into *disentangled* component features. The goal is to learn a dictionary of vectors $\{\mathbf{f}_k\}_{k=1}^{n_{\text{feat}}} \subset \mathbb{R}^d$ that can represent the unknown, ground truth network features as sparse linear combinations. If successful, the learned dictionary contains *monosemantic* neurons corresponding to *features* ([Sharkey et al., 2022b](#)). The autoencoder architecture consists of an encoder and a ReLU activation function, expanding the input dimensionality to $d_{\text{hid}} > d_{\text{in}}$. The encoder’s output is given by:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}), \quad (1)$$

$$\mathbf{x}' = \mathbf{W}_{\text{dec}}\mathbf{h} = \sum_{i=0}^{d_{\text{hid}}-1} h_i \mathbf{f}_i, \quad (2)$$

where $\mathbf{W}_{\text{enc}}, \mathbf{W}_{\text{dec}}^{\top} \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{hid}}}$. The parameter matrix $\mathbf{W}_{\text{dec}}$ forms the feature dictionary, with rows $\mathbf{f}_i$ as dictionary features. The autoencoder is trained to minimize the loss, where the L^1 penalty on $\mathbf{h}$ encourages sparse reconstructions using the dictionary features,

$$\mathcal{L}(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}'\|_2^2 + \alpha \|\mathbf{h}\|_1. \quad (3)$$

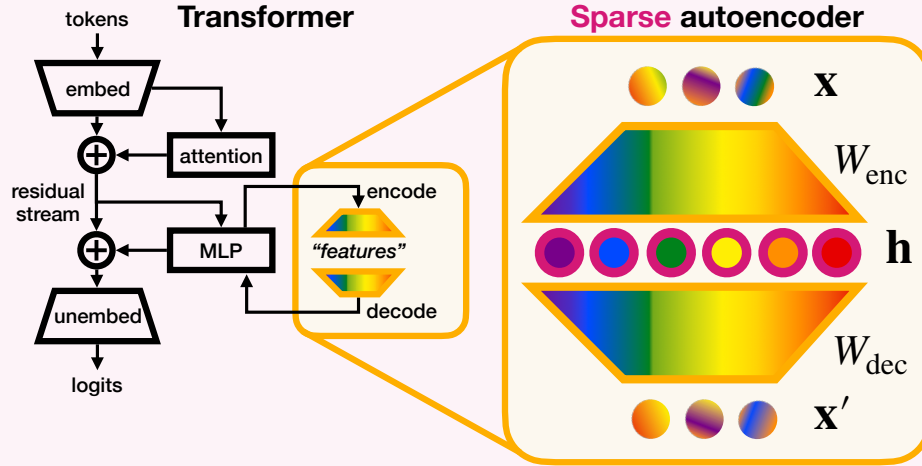


Figure 8: Illustration of a sparse autoencoder applied to the MLP layer activations, consisting of an encoder that increases dimensionality while emphasizing sparse representations and a decoder that reconstructs the original activations using the learned feature dictionary.

4.3 Intervention

Causality as a Theoretical Foundation. The theory of causality ([Pearl, 2009](#)) provides a mathematically precise framework for mechanistic interpretability, offering a rigorous approach to understanding high-level semantics in neural representations ([Geiger et al., 2023a](#)). By treating neural networks as causal models,

with their *compute graphs serving as causal graphs*, researchers can perform precise interventions and examine the roles of individual parameters (Mueller et al., 2024). This causal perspective on interpretability has led to the development of various intervention techniques, including activation patching (Section 4.3.1), causal abstraction (Section 4.3.2), and hypothesis testing methods (Section 4.3.3).

4.3.1 Activation Patching

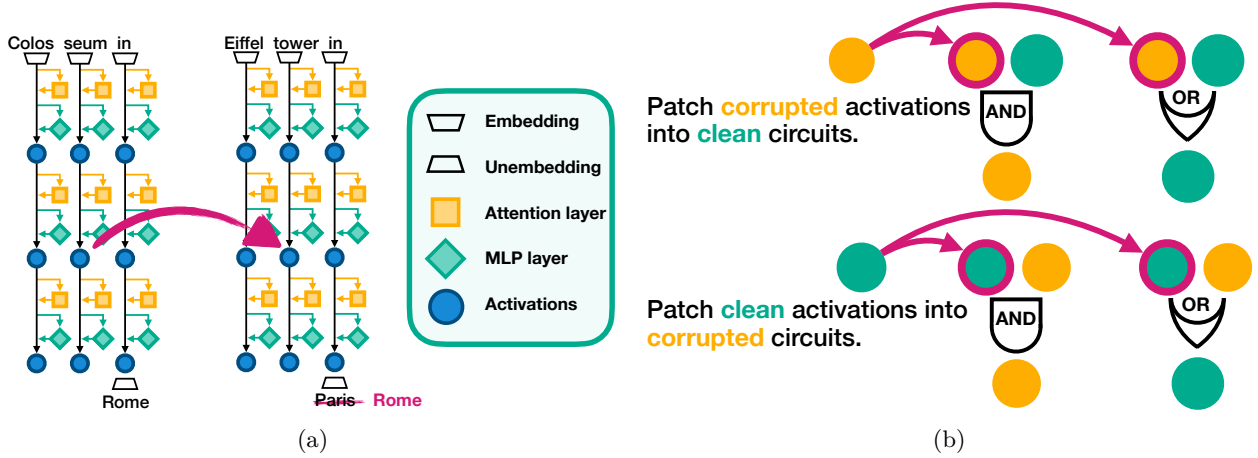


Figure 9: (a) Activation patching in a transformer model. Left: The model processes the clean input "Colosseum in Rome," caching the latent activations (step *i.*). Right: The model runs with the corrupted input "Eiffel Tower in Paris" (step *ii.*). The pink arrow shows an MLP layer activation (green diamond) patched from the clean run into the corrupted run (step *iii.*). This causes the prediction to change from "Paris" to "Rome," demonstrating how the significance of the patched component is determined (step *iv.*). By comparing these carefully selected inputs, researchers can control for confounding circuitry and isolate the specific circuit responsible for the location prediction behavior. (b) Activation patching directions: Top: Patching corrupted activations (orange) into clean circuits (turquoise) reveals *sufficient* components for identifying OR logic scenarios. Bottom: Patching clean activations (green) into corrupted circuits (orange) reveals *necessary* components that are useful for identifying AND logic scenarios. The AND and OR gates demonstrate how these patching directions uncover different logical relationships between model components.

Activation patching is a collective term for a set of causal intervention techniques that manipulate neural network activations to shed light on the decision-making processes within the model. These techniques, including causal tracing (Meng et al., 2022a), interchange intervention (Geiger et al., 2021b), causal mediation analysis (Vig et al., 2020), and causal ablation (Wang et al., 2023), share the common goal of modifying a neural model’s internal state by replacing specific activations with alternative values, such as zeros, mean activations across samples, random noise, or activations from a different forward pass (Figure 9a).

The primary objective of activation patching is to isolate and understand the role of specific components or circuits within the model by observing how changes in activations affect the model’s output. This enables researchers to infer the function and importance of those components. Key applications include localizing behavior by identifying critical activations, such as understanding the storage and processing of factual information (Meng et al., 2022a; Geva et al., 2023; Goldowsky-Dill et al., 2023; Stolfo et al., 2023), and analyzing component interactions through circuit analysis to identify sub-networks within a model’s computation graph that implement specified behaviors (Wang et al., 2023; Hanna et al., 2023; Lieberum et al., 2023; Hendel et al., 2023; Geva et al., 2023).

The standard protocol for activation patching (Figure 9a) involves:

- step *i.* Running the model with a clean input and caching the latent activations;
- step *ii.* Executing the model with a corrupted input;

- step *iii*. Re-running the model with the corrupted input but substituting specific activations with those from the clean cache; and
- step *iv*. Determining significance by observing the variations in the model’s output during the third step, thereby highlighting the importance of the replaced components.

This process relies on comparing pairs of inputs: a clean input, which triggers the desired behavior, and a corrupted input, which is identical to the clean one except for critical differences that prevent the behavior. By carefully selecting these inputs, researchers can *control for confounding circuitry* and isolate the specific circuit responsible for the behavior.

Differences in patching direction – clean to corrupted (causal tracing) versus corrupted to clean (resample ablation) – provide insights into the sufficiency or necessity of model components for a given behavior. Clean to corrupted patching identifies activations sufficient for restoring clean performance, even if they are unnecessary due to redundancy, which is particularly informative in OR logic scenarios (Figure 9b, OR gate). Conversely, corrupted to clean patching determines the necessary activations for clean performance, which is useful in AND logic scenarios (Figure 9b, AND gate).

Activation patching can employ corruption methods, including zero-, mean-, random-, or resample ablation, each modulating the model’s internal state in distinct ways. Resample ablation stands out for its effectiveness in maintaining consistent model behavior by not changing the data distribution too much (Zhang & Nanda, 2023). However, it is essential to be careful when interpreting the patching results, as breaking behavior by taking the model off-distribution is uninteresting for finding the relevant circuit (Nanda, 2023e).

Path Patching and Subspace Activation Patching. Path patching extends the activation patching approach to multiple edges in the computational graph (Wang et al., 2023; Goldowsky-Dill et al., 2023), allowing for a more fine-grained analysis of component interactions. For example, path patching can be used to estimate the direct and indirect effects of attention heads on the output logits. Subspace activation patching, also known as distributed interchange interventions (Geiger et al., 2023b), aims to intervene only on linear subspaces of the representation space where *features* are hypothesized to be encoded, providing a tool for more targeted interventions.

Recently, Ghandeharioun et al. (2024) introduced *patchscopes*, a framework that unifies and extends activation patching techniques: using the model’s text generation to explain internal representations, it enables more flexible interventions across various interpretability tasks, improving early layer inspection and allowing for cross-model analysis.

Limitations and Advancements. Activation patching has several limitations, including the effort required to design input templates and counterfactual datasets, the need for human inspection to isolate important subgraphs, and potential second-order effects that can complicate the interpretation of results (Lange et al., 2023) and the *hydra effect* (McGrath et al., 2023; Rushing & Nanda, 2024) (see discussion in Section 7.2). Recent advancements aim to address these limitations, such as automated circuit discovery algorithms (Conmy et al., 2023), gradient-based methods for scalable component importance estimation like attribution patching (Nanda, 2023d; Syed et al., 2023), and techniques to mitigate self-repair interference during analysis (Ferrando & Voita, 2024).

4.3.2 Causal Abstraction

Causal abstraction (Geiger et al., 2021a; 2023a) provides a mathematical framework for mechanistic interpretability, treating neural networks and their explanations as causal models. This approach validates explanations through interchange interventions on network activations (Jenner et al., 2023), unifying various interpretability methods such as LIME (Ribeiro et al., 2016), causal effect estimation (Feder et al., 2021), causal mediation analysis (Vig et al., 2020), iterated nullspace projection (Ravfogel et al., 2020), and circuit-based explanations (Geiger et al., 2023a).

To overcome computational limitations, *distributed alignment search* (Geiger et al., 2023b) introduced gradient-based distributed interchange interventions, extending causal abstraction to larger models (Wu